Sport Data Solution

POC Avantages Sportifs

Alexandre — Data Engineer

Soutenance P12 — 2026

Contexte & Objectifs

Sport Data Solution : startup fondée par Alexandre (cycliste) et Juliette (marathonienne). Objectif : récompenser les salariés sportifs et encourager la pratique du sport au sein de l'entreprise.

Prime Sportive — 5%

- Salariés venant au bureau en faisant du sport
- Marche : max 15 km / Vélo : max 25 km
- Validation via Google Maps API
- Déclaration validée par le salarié

5 Journées Bien-être

- Activité physique régulière hors travail
- Minimum 15 activités par an (12 mois glissants)
- Données simulées (futur : Strava API)
- Notification Slack automatique

Architecture Globale

Phase 1 — RH

Excel → Pydantic → Transform
→ Google Maps → pgcrypto
→ Upsert employees

Phase 2 — Activités

Generate → Redpanda
→ Consumer PostgreSQL
→ Consumer Slack

Phase 3 — Bénéfices

Read rules (SCD2)
→ Compute éligibilité
→ Upsert employee_benefits

PostgreSQL 15.7 | Redpanda 24.2 | Kestra 0.18 | Metabase 0.50 | Python 3.11

Orchestration : Kestra — 4 flows YAML | Docker Compose — 5 containers sur sport_network

employees

161 rows — PII chiffre (pgcrypto)

sport_activities

7 240 rows — 12 mois historique

benefit_rules

5 rows — SCD2 (paramètres)

employee_benefits

161 rows — Résultats calculés

Choix Techniques & Justifications

Composant	Choix	Alternative	Justification
Broker	Redpanda	Apache Kafka	Plus simple (pas de ZooKeeper), API Kafka-compatible, ARM natif
Client Kafka	confluent-kafka	kafka-python	kafka-python : échecs silencieux avec Redpanda
Orchestrateur	Kestra (OSS)	Airflow	YAML natif, UI intégrée, léger, pas de boilerplate Python
Dashboard	Metabase	Streamlit	Zero code, filtres natifs, requêtes SQL, rafraîchissement auto
Base de données	PostgreSQL	DuckDB / SQLite	Multi-utilisateur, pgcrypto, production-grade
Chiffrement	pgcrypto	Applicatif	Natif PostgreSQL, Metabase peut dechiffrer en SQL
Paramètres	SCD2 (INSERT)	UPDATE en place	Historique complet, audit, demo live
Suppression	Soft delete	Hard delete	Réversible, traçabilité, compatible audit

Pipeline de Données — Détail

Phase 1 — Pipeline RH (load_employees.py)

Extract Excel (Pandas) → Validate (Pydantic, 11 champs) → Transform (mapping transport FR→ENUM) → Google Maps API (parse adresse + calcul distance domicile-bureau) → Encrypt PII (pgcrypto, 8 colonnes) → Upsert PostgreSQL (ON CONFLICT + soft delete)

Phase 2 — Pipeline Activités (generate + publish + consume)

Generate 7 237 activités (seed=42, déterministe) → Publish JSON vers Redpanda topic → Consumer 1 : INSERT dans sport_activities (PostgreSQL) → Consumer 2 : notification Slack (message de félicitations)

Phase 3 — Pipeline Bénéfices (compute_benefits.py)

Lecture regles actives (DISTINCT ON, SCD2) → Jointure employees + activities (12 mois glissants) → Calcul prime (salaire * taux) + bien-être (count >= seuil) → Upsert employee_benefits (idempotent)

Qualité des Données — 4 Couches

Couche 1 — Pydantic (ligne par ligne)

Validation des types, plages, règles croisées (age >= 16, max_speed >= avg_speed). Avant toute insertion en base.

Couche 2 — PostgreSQL (contraintes DB)

ENUMs, NOT NULL, UNIQUE, FK, CHECK. Protection au niveau base de données contre les données invalides.

Couche 3 — Great Expectations (dataset)

31 expectations après chaque run du pipeline. Complétude, unicité, plages de valeurs. Rapport HTML généré automatiquement.

Couche 4 — Pytest (code)

76 tests unitaires couvrant la logique de transformation, le calcul d'éligibilité et les validateurs Pydantic.

Sécurité & Protection des Données

Chiffrement PII

8 colonnes chiffrées (nom, prénom, salaire, adresse, date naissance) via pgcrypto. Chiffrement symétrique AES, clé dans .env (gitignored).

3 Rôles PostgreSQL

sport_admin (DDL) / sport_writer (DML, pipeline uniquement) / sport_reader (SELECT, Metabase). Principe du moindre privilège.

Isolation Réseau

Tous les services communiquent sur un bridge Docker isolé (sport_network). Aucun trafic ne quitte la machine. Pas de SSL en POC (justifié : réseau local).

Gestion des Secrets

Variables d'environnement dans .env (gitignored). .env.example documente les variables requises. Kestra : injection via SPORT_* prefix.

Monitoring & Orchestration

4 Flows Kestra

Flow	Description	Déclenchement
full_pipeline	Exécute les 3 phases séquentiellement	Manuel / Schedule
recompute_benefits	Phase 3 uniquement (après changement de paramètres)	Manuel
update_benefit_rules	INSERT SCD2 + recompute automatique	Demo live
inject_live_activity	N activités → Redpanda → Slack → DB → Recompute	Demo live

Supervision continue

- Kestra UI : timeline d'exécution, statut SUCCESS/FAILED, logs par tâche
- Great Expectations : rapport HTML après chaque pipeline run (31 checks)
- Python logging : fichiers journaliers rotatifs + console (src/utils/logger.py)

Dashboard Metabase — KPIs

126 / 161

Employés éligibles

78% des effectifs

172 482 EUR

Coût prime totale

Taux actuel : 5%

475 jours

Jours bien-être

95 employés x 5 jours

~2%

Coût / masse salariale

Impact maîtrisé

Autres indicateurs sur le dashboard :

- Répartition du coût par BU (bar chart) — filtrable via dropdown
- Distribution des sports pratiqués (bar chart horizontal) — top activités
- Tendance des activités par mois (line chart) — saisonnalité visible
- Historique des paramètres (table SCD2) — traçabilité des changements
- Simulateur de coût (prime rate + well-being days ajustables) — sans relancer le pipeline

3 filtres dashboard : Business Unit | Prime Rate (simulation) | Well-Being Days (simulation)

Demo Live & Conclusion

Demo 1 — Injection d'activités

1. Ouvrir Metabase + Kestra
2. Exécuter inject_live_activity (N=3)
3. Voir 3 messages Slack
4. Rafraîchir Metabase : KPIs mis a jour

Demo 2 — Changement de paramètres

1. Montrer coût actuel : 172 482 EUR (5%)
2. Exécuter update_benefit_rules (0.07)
3. Pipeline recompute automatique
4. Nouveau coût : ~241 475 EUR (7%)

Résultats Clés

161	7 240	126	4	76
employés chargés	activités générées	employés éligibles	couches qualite	tests unitaires

Merci — Questions ?